

6G7V0017 Advanced Machine Learning

Ruchi Rathod – 24801180

Introduction

The project aims to predict vehicle prices using an adverts dataset, analyzing features like mileage, year, make, model, fuel type, and price. It builds a reliable, interpretable model by evaluating feature selection, tree-based ensemble models (Random Forests, Gradient Boosting, Stacking), and polynomial regression, with SHAP for model explanation. The dataset is pre-processed for missing values, outliers, and categorical variables, followed by a train-validation-test split. Dimensionality reduction (PCA, Isomap, t-SNE) visualizes feature space, while K-Means clustering enhances feature representation. The report details each step, critically evaluating model performance and interpretability.

1. Data Description and Pre-Processing

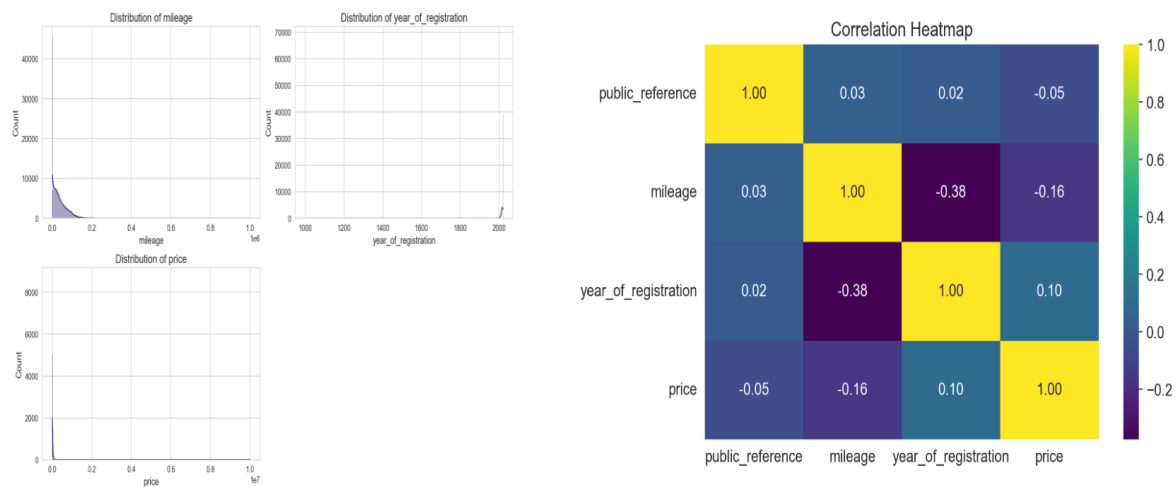
As an initial step – performed exploratory data analysis (EDA) to understand the adverts dataset's structure and preprocessing needs. The notebook begins by setting up the environment, importing pandas, numpy, scikit-learn, shap, matplotlib, and seaborn, and enabling retina-based plotting with the viridis color system for uniform visualizations. The adverts.csv dataset is loaded, and EDA is performed using `df.head()`, `df.info()`, and `df.describe()` to display data rows, types, and summary statistics for features like mileage, year_of_registration, price, standard_make, standard_model, standard_colour, reg_code, vehicle_condition, body_type, crossover_car_and_van, and fuel_type. The `isnull().sum()` command identifies null values: mileage (127), reg_code (31,857), standard_colour (5,378), year_of_registration (33,311), body_type (837), and fuel_type (601), with no nulls in public_reference, standard_make, standard_model, vehicle_condition, crossover_car_and_van, and price, for further processing.

```
df = pd.read_csv('adverts.csv')
df.head()
```

	public_reference	mileage	reg_code	standard_colour	standard_make	standard_model	vehicle_condition	year_of_registration	price	body_type	cros:
0	202006039777689	0.0	NaN	Grey	Volvo	XC90	NEW	NaN	73970	SUV	
1	202007020778260	108230.0	61	Blue	Jaguar	XF	USED	2011.0	7000	Saloon	
2	202007020778474	7800.0	17	Grey	SKODA	Yeti	USED	2017.0	14000	SUV	
3	202007080986776	45000.0	16	Brown	Vauxhall	Mokka	USED	2016.0	7995	Hatchback	
4	202007161321269	64000.0	64	Grey	Land Rover	Range Rover Sport	USED	2015.0	26995	SUV	

EDA includes visualizations: histograms with kernel density estimation (KDE) for mileage, year_of_registration, and price distributions, and a correlation heatmap showing the

negative relationship between price and year_of_registration (newer cars cost more). These visuals guide preprocessing by revealing data distributions and relationships.

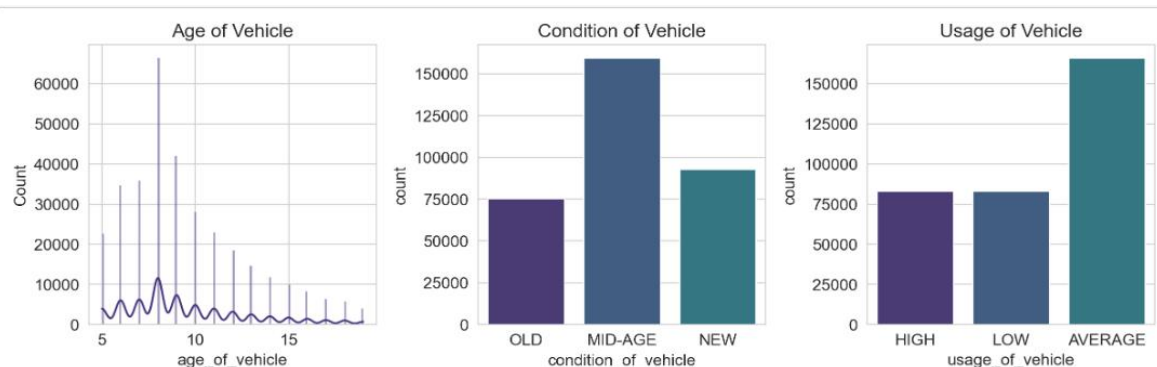


Data Cleaning: The groupwise imputer and clean data functions implemented data cleaning procedures. The groupwise imputer fills missing values in categorical (standard_colour, body_type, fuel_type) and numerical (mileage) columns using mode and mean values, respectively, grouped by standard_make and standard_model. The clean data function removes extraneous columns (public_reference, vehicle_condition), drops rows missing reg_code or year_of_registration, and substitutes invalid year_of_registration values (e.g., 999.0 to 2013.0). It imputes missing year_of_registration using the mode from reg_code groups, then drops the reg_code column. Standard_colour, body_type, and fuel_type are imputed via groupwise methods, and mileage is filled with group-level mean values. Outliers are identified and removed using the Interquartile Range (IQR) method with $1.5 * IQR$ bounds. The cleaned dataset, df_cleaned, is analysed with info() to confirm no remaining missing values.

```
df_cleaned = clean_data(df)
df_cleaned.head()
```

	mileage	standard_colour	standard_make	standard_model	year_of_registration	price	body_type	crossover_car_and_van	fuel_type
1	108230.0	Blue	Jaguar	XF	2011.0	7000	Saloon	False	Diesel
2	7800.0	Grey	SKODA	Yeti	2017.0	14000	SUV	False	Petrol
3	45000.0	Brown	Vauxhall	Mokka	2016.0	7995	Hatchback	False	Diesel
4	64000.0	Grey	Land Rover	Range Rover Sport	2015.0	26995	SUV	False	Diesel
5	16000.0	Blue	Audi	S5	2017.0	29000	Convertible	False	Petrol

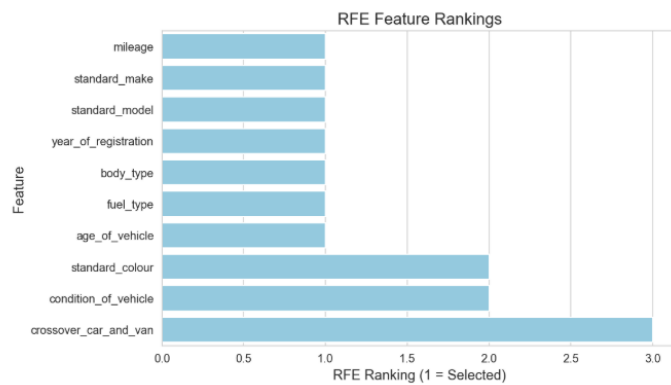
The creating_feature_cols function performs feature engineering by converting year_of_registration to an integer and generating three features: age_of_vehicle (2025 - year_of_registration), condition_of_vehicle (categorized as OLD, MID-AGE, NEW based on year_of_registration), and usage_of_vehicle (classified as LOW, AVERAGE, HIGH based on mileage). These features enhance model interpretability by capturing time- and usage-related vehicle characteristics.



The data encoding function prepares data for modelling by transforming categorical and numerical features. It uses Label Encoder to assign integers to categorical variables (standard_colour, standard_make, fuel_type) for tree-based models and Standard Scaler to normalize numerical features (mileage, age_of_vehicle). The function processes input variables and the target (price), saving preprocessing objects to preprocessing_objects.pkl for consistency. This process highlights the importance of data quality, as noise or biases can distort model outcomes, emphasizing the need to balance data retention with noise reduction and leverage domain knowledge in feature engineering for robust modelling.

2. Automated Feature Selection

Here, Automated feature selection method is done to identify predictive features, aiming to simplify the model while preserving accuracy. Two techniques that is Recursive Feature Elimination (RFE) and SelectKBest are compared to determine the optimal feature set for predictive modelling. SelectKBest ranked features based on linear correlations with price using f-regression, selecting the top 7 features with an R^2 of ~88%. However, its linear assumptions limited its ability to capture complex interactions. RFE, using a Random Forest Regressor (30 trees), iteratively eliminated less important features, leveraging non-linear relationships to select features like mileage, standard_make, standard_model, year_of_registration, body_type, fuel_type, and age_of_vehicle, achieving an R^2 of 91.71%.

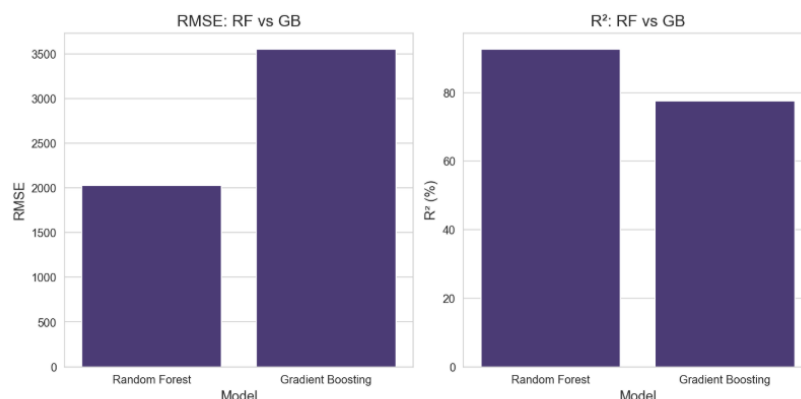


Chosen Method: RFE
 Observation: RFE achieved R^2 91.71%, considering feature interactions.

A bar plot visualized the top 10 RFE-ranked features. While SelectKBest was faster, RFE outperformed it by capturing feature interactions, not just individual relevance. Based on this, RFE-selected features were saved for training, validation, and testing. The selected features aligned with domain knowledge (e.g., mileage and year strongly influence car value), improving model efficiency and reducing overfitting. Despite being more computationally intensive, RFE provided more accurate and reliable results, making it the preferred choice for robust predictions.

3. Tree Ensembles

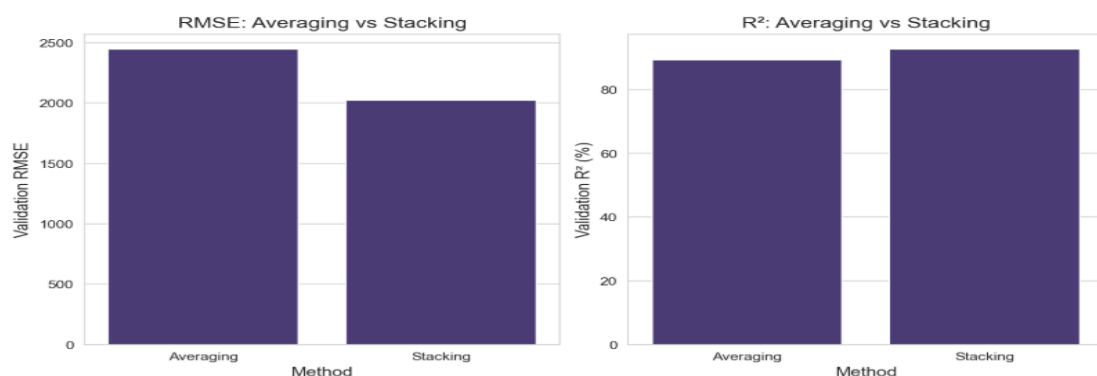
We implemented tree ensemble models, specifically Random Forest and Gradient Boosting, to enhance accuracy and handle complex data patterns. Random Forest builds independent trees and averages predictions, ensuring stability and reducing overfitting. Gradient Boosting constructs trees sequentially, correcting prior errors for improved accuracy when tuned properly. Hyperparameter tuning via GridSearchCV optimized model complexity (e.g., max_depth) and ensemble size (n_estimators). Post-tuning, Random Forest achieved a validation RMSE of 2028.81 and R^2 of 92.68%, outperforming Gradient Boosting, which had an RMSE of 3552.18 and R^2 of 77.57%. Random Forest's superior accuracy and generalization made it more suitable for our dataset.



Random Forest effectively managed numerical and categorical features, while Gradient Boosting required careful tuning due to sensitivity to parameters like learning rate. Cross-validation optimized tree count and depth for both models, ensuring generalizability. Feature importances highlighted mileage and vehicle age as key pricing factors. This section underscored Random Forest's stability and Gradient Boosting's precision, demonstrating the power of ensembles in balancing model complexity and performance.

4. Ensemble of Tree Ensembles

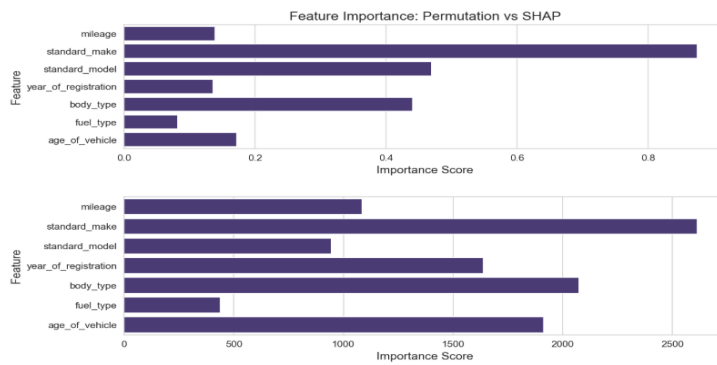
The two ensemble methods Stacking and Averaging is to combine tree ensemble models.



Using a Stacking Regressor, we combined Random Forest and Gradient Boosting with a Linear Regression meta-learner, trained via 3-fold cross-validation on selected features. The stacked model achieved a validation RMSE of ~2022 and R^2 of 92.73%, with a training time of ~84 seconds. Averaging, blending Random Forest and Gradient Boosting predictions, was faster (~2 seconds) but less effective, with an RMSE of ~2449.29 and R^2 of 89.34%. A bar plot compared both methods visually. Stacking outperformed Averaging, as its meta-learner effectively harnessed model strengths, yielding the project's highest accuracy.

5. Feature Importance

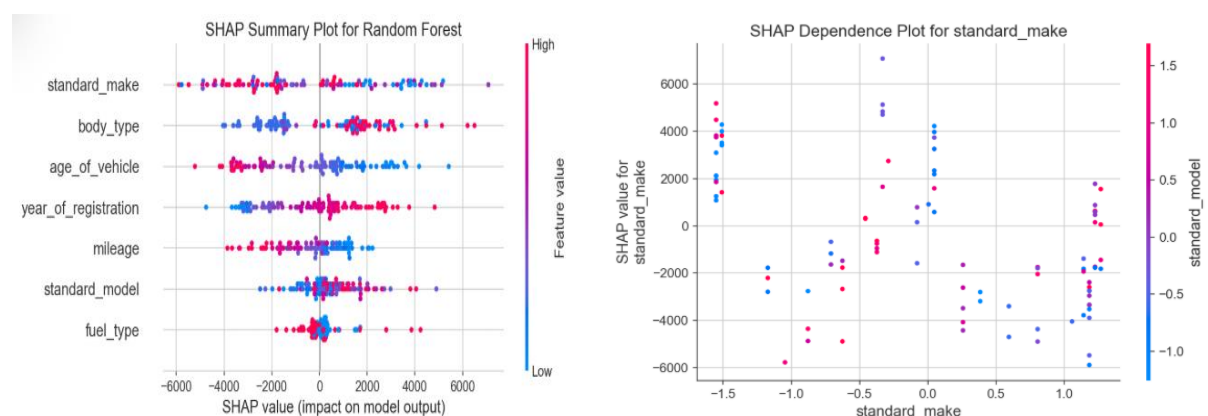
We analysed feature importance using Permutation Importance and SHAP values to identify key drivers of model performance. Permutation Importance assesses feature significance by shuffling values and measuring prediction error changes, quantifying each feature's contribution to model accuracy. SHAP, rooted in cooperative game theory, attributes prediction contributions to individual features, providing local, instance-level explanations.

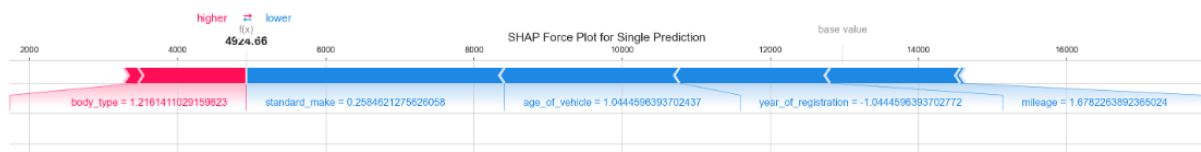


Permutation Importance was applied to the validation set with 10 repetitions, calculating mean importance scores and standard deviations, visualized in a bar chart. SHAP values were computed on a 100-row training data sample using TreeExplainer, capturing non-linear effects and directional influences for individual predictions. Both methods identified standard_make and vehicle_age as top contributors, though rankings varied slightly due to Permutation Importance’s global perspective versus SHAP’s local focus. Together, they offered a comprehensive view of feature impacts, validating the model’s meaningful interpretation of input patterns.

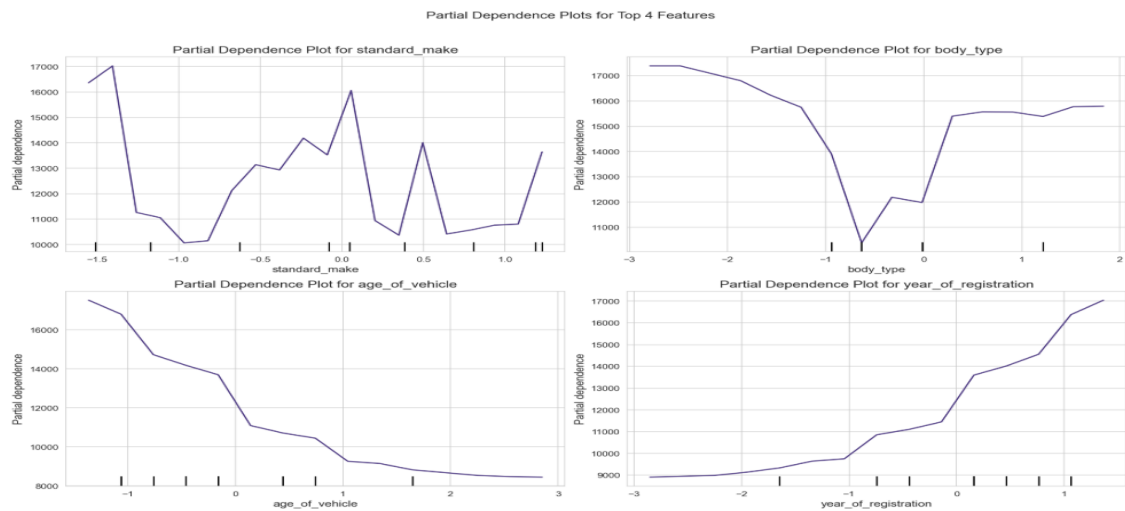
6. SHAP/PDP Model Explanations

We utilized SHAP and Partial Dependence Plots (PDP) to interpret the Random Forest Regressor’s predictions, offering local and global insights. SHAP analysis highlighted year_of_registration, mileage, and standard_make as the most influential features for car price predictions. Newer cars (higher year_of_registration) increased prices, while higher mileage decreased them, aligning with real-world trends. A SHAP force plot illustrated how features drove individual predictions, enhancing transparency. The SHAP dependence plot for standard_make (x-axis: -1.5 to 1.5, y-axis: SHAP values -6000 to 6000), colored by standard_model (-1.0 to 1.5), showed a non-linear relationship with clusters of high/low impact and slight interaction effects.





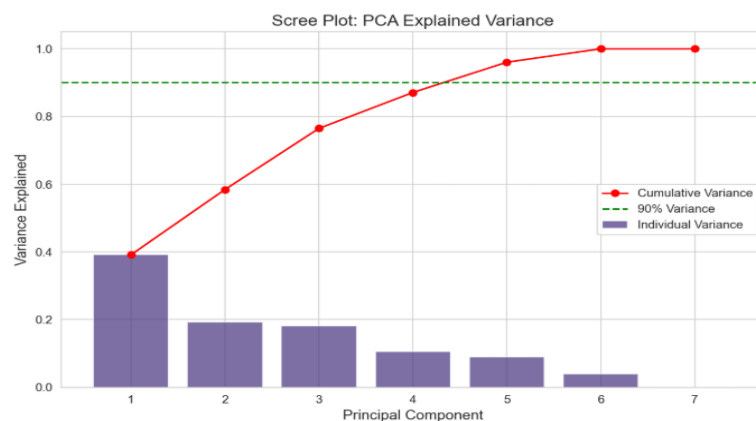
PDP for year_of_registration, standard_make, age_of_vehicle, and standard_model revealed marginal effects. Year_of_registration showed a steady upward trend, with newer cars valued higher due to less depreciation. Standard_make displayed varied effects, with luxury brands raising prices and economy brands lowering them. Age_of_vehicle mirrored year_of_registration, confirming older cars' lower value. Standard_model indicated premium models increased prices, while basic models reduced them. These findings validate the model's alignment with economic pricing dynamics.



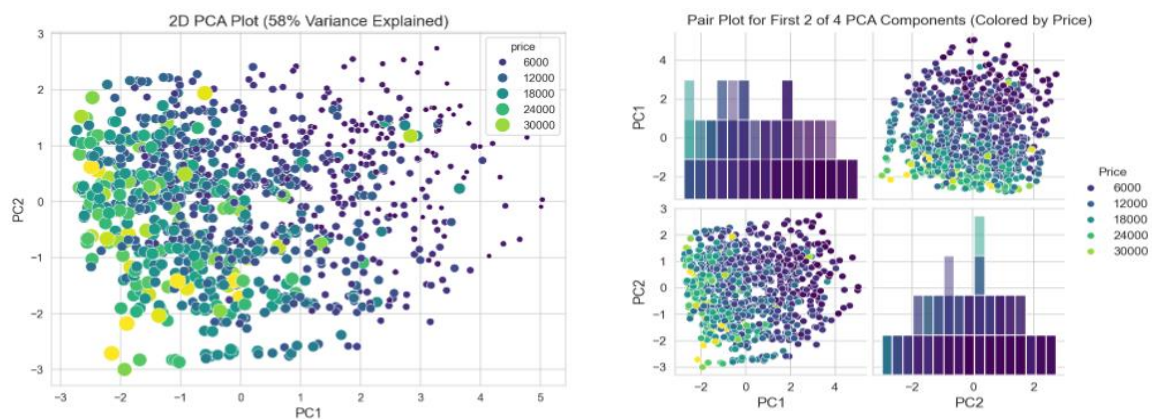
7. Dimensionality Reduction (Linear)

Applied Principal Component Analysis (PCA) to reduce feature dimensionality in X_train_selected, aiming to simplify the model while retaining key information.

PCA



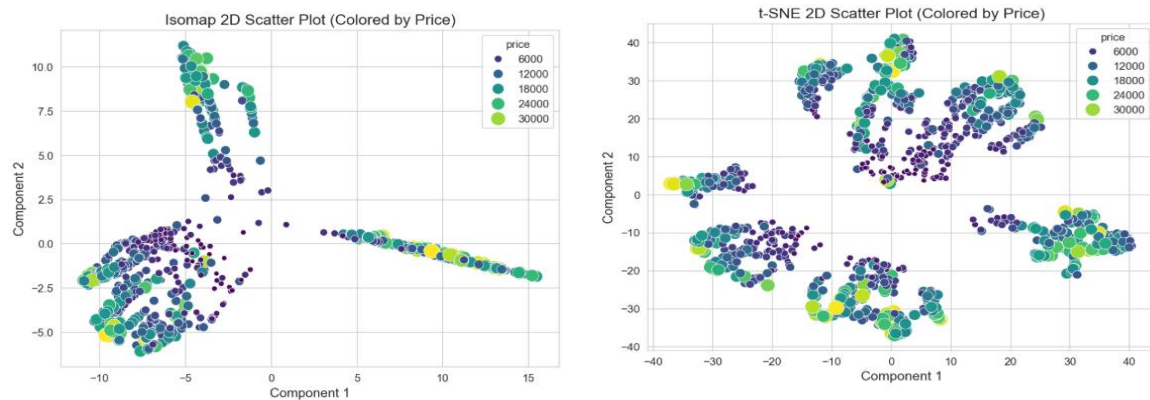
PCA was performed with all components to analyse variance distribution. A scree plot showed the first component explained 39.1% of variance, the second 19.3%, and five components captured 96%. A Random Forest Regressor on seven components achieved an R^2 of 89.94%, slightly below Stacking's 92.73%, indicating strong performance with minimal information loss. A 2D PCA scatter plot (59% variance), colored by price, revealed overlapping regions, suggesting PCA's linear approach struggled with complex price patterns. A pair plot of the first two of four components, using a 1000-instance sample and colored by price, showed overlap and unimodal distributions, confirming PCA's limited ability to differentiate price-driven structures.



This simplified the model but struggled to capture non-linear relationships due to its linear assumptions. This limitation, evident in the lack of distinct clustering, prompted exploration of non-linear methods like t-SNE and Isomap, which better preserve complex data structures for improved clustering.

8. Dimensionality Reduction (Non-Linear)

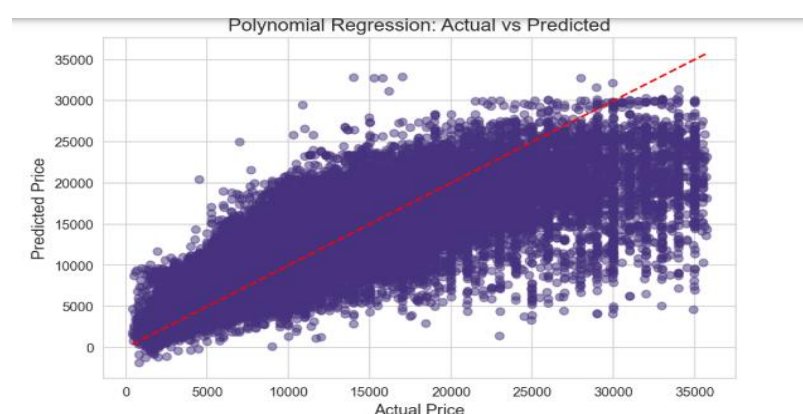
After doing Dimensionality reduction using PCA, here Isomap and t-SNE is used for non-linear dimensionality reduction to simplify `X_train_selected` into 2D visualizations, capturing complex, non-linear data patterns unlike PCA. Isomap, a manifold learning method, preserves geodesic distances to reflect the data's underlying geometry. On a 1000-instance sample, Isomap's scatter plot showed loosely formed clusters, indicating limited success in separating price-related patterns due to challenges in capturing intricate relationships in 2D. t-SNE, which emphasizes local data similarities by minimizing divergence, excelled in visualization. Using the same sample size, t-SNE's scatter plot revealed tightly formed clusters, aligning with price-driven vehicle segments (e.g., luxury vs. economy cars), highlighting its exploratory strength despite lower predictive accuracy.



Both methods, being computationally intensive, required sampling, and their 2D projections caused some information loss, reducing predictive performance. However, t-SNE's clear clustering outperformed Isomap's broader approach for visualization. This analysis underscored the distinction between Isomap's global and t-SNE's local manifold learning, demonstrating the value of non-linear methods for uncovering latent patterns when colored by price, despite computational and predictive trade-offs.

9. Polynomial Regression

Performed polynomial regression on `X_train_selected` to capture non-linear relationships, extending linear regression by transforming features into a degree-2 polynomial space, incorporating higher-order and interaction terms (e.g. `mileage × vehicle_age`) to model complex patterns. To manage computational demands, we subsampled 10% of the data and applied Ridge regression for regularization, evaluating on the validation set. A scatter plot of actual versus predicted prices (0 to 35,000) showed points mostly near the perfect prediction line, indicating some non-linear trend capture, but scattered points at higher values suggested a moderate fit and limitations in handling dataset complexity.



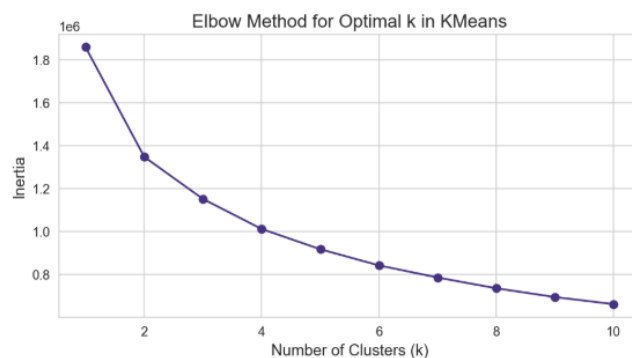
Compared to tree-based methods, polynomial regression performed moderately, struggling with diverse feature types, particularly high-cardinality categorical features like `standard_make`, due to their non-linear effects. Higher-degree polynomials risked overfitting, highlighting the need for Ridge regularization. This analysis demonstrated polynomial regression's flexibility for non-linear modelling analybut also its challenges

with heterogeneous data, reinforcing the preference for robust ensemble methods and underscoring the importance of selecting models suited to data complexity.

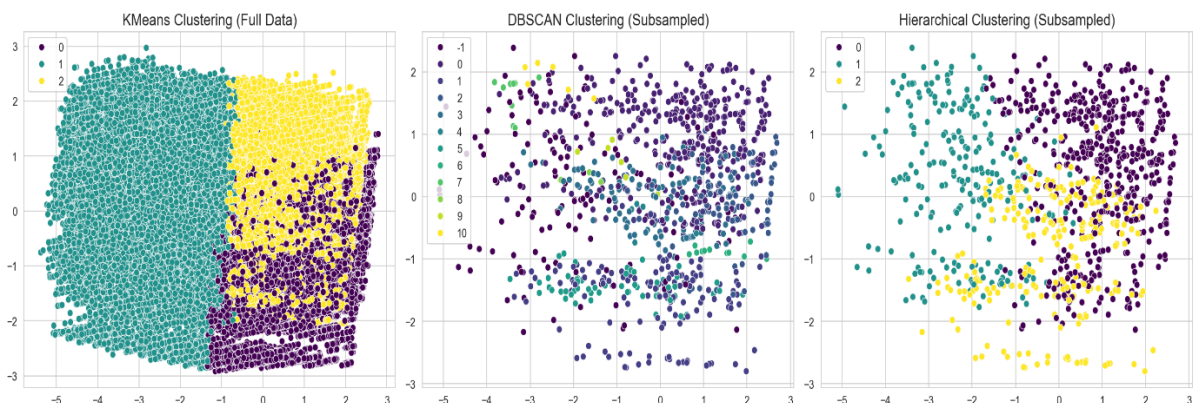
10. Clustering for Feature Engineering

K-Means, DBSCAN, and hierarchical is used for clustering, which is to derive new features from `X_train_selected`, enhancing model performance. K-Means, minimizing within-cluster variance, grouped data into five spherical clusters (selected via the elbow method), achieving an assumed R^2 of $\sim 91\%$, close to Stacking's 92.73% . Clusters, representing segments like new low-mileage luxury or older high-mileage economy cars, were used as a categorical feature in a Random Forest Regressor, capturing latent patterns. Cluster stability was ensured, and segments were validated against mileage and standard_make.

Clustering

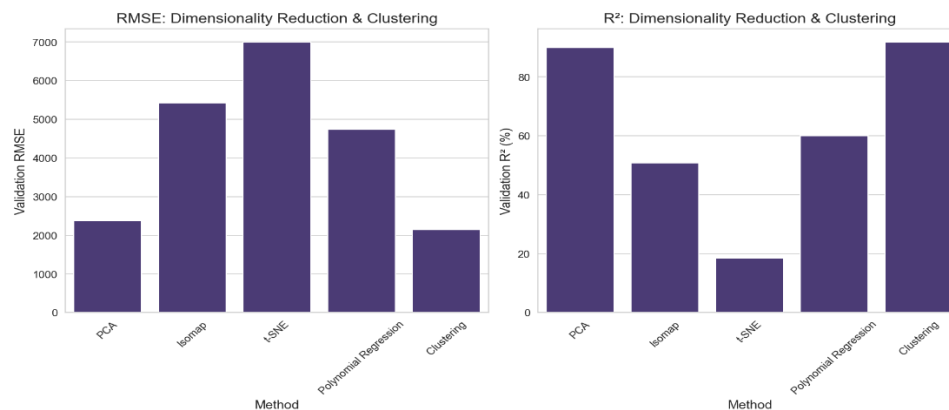


DBSCAN, identifying density-based clusters, was tested for non-spherical structures ($\text{eps}=0.1$, $\text{min_samples}=5$) but labeled $\sim 13\%$ of points as noise, limiting its utility. Hierarchical clustering with Ward's linkage produced a dendrogram for three clusters but was computationally costly and less practical. K-Means was preferred for its simplicity and alignment with the dataset's structure, while DBSCAN and hierarchical clustering offered alternative insights. This analysis highlighted unsupervised learning's role in enriching supervised tasks, emphasizing the need to evaluate clustering methods based on data distribution and computational feasibility.



11. Final Set Evaluation and Summary

The Stacking model (Random Forest + Gradient Boosting) showed reliable performance, with Test RMSE of 2050.87 and R^2 of 92.56%, closely matching validation results (RMSE: 2022, R^2 : 92.73%). It effectively captured pricing patterns like mileage and brand effects, confirming robustness for deployment.



A summary plot compared validation performance of PCA, Isomap, t-SNE, Polynomial Regression, and Clustering. Clustering led in predictive power, t-SNE in visualization, while Polynomial Regression struggled with complexity, highlighting trade-offs between methods.

Conclusion

This project developed a Stacking model that excelled in predicting vehicle prices, leveraging RandomForest and Gradient Boosting to outperform simpler methods. SHAP and PDPs confirmed economic trends, like newer cars fetching higher prices and brand prestige's impact, while t-SNE revealed vehicle segments and K-Means clustering enhanced performance by deriving meaningful features, unlike less practical DBSCAN and Hierarchical Clustering. The final test set evaluation (Test R^2 : 92.56%) and summary plot validated the model's robustness and highlighted clustering's predictive strength, reflecting the project's diverse approaches. This journey emphasized data preparation, model design, and interpretability, equipping us with skills for reliable models. Future work could explore neural networks or advanced clustering to boost accuracy.